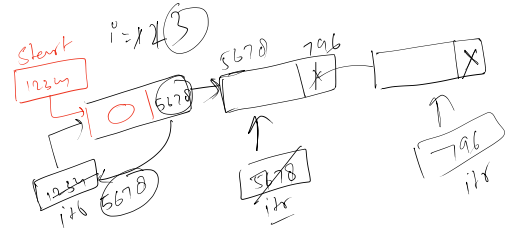


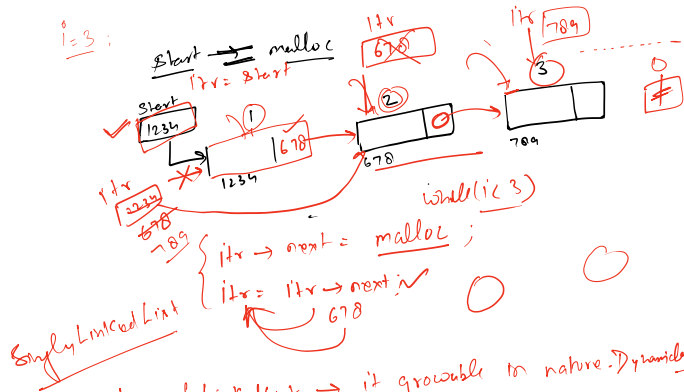
`struct Student * createList (int n)
 {
 int i=0, * struct Student * Start;
 if (n <= 0) return NULL;
 i=1; while (i <= n)
 {
 Start = (struct Student *) malloc (sizeof (struct Student));
 Start->next = Start;
 i++;
 }
 return Start;
 }`

Void pointer
 i=1 while (i <= 3)



`itr->next = malloc;
 itr++;
 itr = itr->next;
 itr->next = NULL;`

`}
 return Start;`



Advantage of Linked List → it growable in nature. Dynamically
 → It is Not Contiguous that's why it doesn't allow to access the Node Randomly.

The complexity to access a value in array is $O(1)$
 But in the Linked List we require n traversal to fetch hence the complexity is $O(n)$

- ① Write a function to Traverse and Print the Singly and Doubly Linked List
- ② Delete a Node which Position?
- ③ Insert a Node at Given Position?
- ④ Search an Element into the List and return the Node Number?

